

07-Python 基础：条件、循环、字符串、列表与函数

上一节学习了 Python 的工具、输出、输入、变量和表达式。本节继续学习编程最常用的结构：条件判断、循环、字符串、列表和函数。这些内容是完成小项目的基础。

一、学习目标

学完本节，你应该能做到：

1. 使用 if、elif、else 编写条件判断。
 2. 理解缩进在 Python 中的重要作用。
 3. 使用 for 和 range() 处理固定次数重复。
 4. 使用 while 处理满足条件时重复。
 5. 对字符串进行长度、拼接、查找等基本操作。
 6. 使用列表保存多个数据。
 7. 遍历列表并进行简单统计。
 8. 定义和调用简单函数。
 9. 使用测试数据检查程序是否正确。
-

二、条件判断

条件判断让程序根据不同情况执行不同代码。

生活例子：

如果下雨，就带伞。
否则，不带伞。

Python 写法：

```
weather = input("今天下雨吗？请输入 yes 或 no：")

if weather == "yes":
    print("记得带伞。")
else:
    print("不用带伞。")
```

1. if 的基本结构

`if` 条件：
 条件成立时执行的代码

注意：

1. `if` 后面是条件。
2. 条件后面要写英文冒号 `:`。
3. 下一行要缩进。
4. 缩进表示这行代码属于 `if` 管理。

2. if else

```
score = int(input("请输入成绩："))  
  
if score >= 60:  
    print("及格")  
else:  
    print("不及格")
```

IPO 表：

项目	内容
输入	成绩 score
处理	判断 score 是否大于等于 60
输出	及格或不及格

3. if elif else

多个等级判断可以用 `elif`。

```
score = int(input("请输入成绩："))  
  
if score >= 90:  
    print("优秀")  
elif score >= 80:  
    print("良好")  
elif score >= 60:  
    print("合格")  
else:  
    print("需要努力")
```

4. 判断顺序很重要

下面写法是错误思路：

```
score = int(input("请输入成绩："))

if score >= 60:
    print("合格")
elif score >= 80:
    print("良好")
elif score >= 90:
    print("优秀")
else:
    print("需要努力")
```

如果输入 95，第一条 `score >= 60` 已经成立，程序会输出“合格”，不会继续判断后面的“良好”和“优秀”。

所以成绩等级要从高到低判断。

5. 比较运算符

运算符	含义	示例
>	大于	<code>score > 60</code>
<	小于	<code>age < 18</code>
>=	大于等于	<code>score >= 60</code>
<=	小于等于	<code>score <= 100</code>
==	等于	<code>answer == "yes"</code>
!=	不等于	<code>name != ""</code>

注意：

- `=` 是赋值。
- `==` 是判断是否相等。

三、缩进

Python 用缩进表示代码层级。缩进不对，程序可能报错或结果不对。

正确：

```
score = int(input("请输入成绩："))

if score >= 60:
    print("及格")
    print("继续保持")
else:
    print("需要努力")
    print("请认真复习")
```

错误：

```
score = int(input("请输入成绩："))

if score >= 60:
    print("及格")
else:
    print("需要努力")
```

问题：if 和 else 管理的代码没有缩进。

缩进自查

- ☐ if、elif、else 后面有英文冒号。
 - ☐ 属于条件内部的代码向右缩进。
 - ☐ 同一层代码左边对齐。
 - ☐ 不要一会儿用 2 个空格，一会儿用 4 个空格。
-

四、for 循环

for 循环适合处理固定次数的重复。

1. 输出 1 到 5

```
for i in range(1, 6):
    print(i)
```

运行结果：

```
1
2
3
4
5
```

注意：range(1, 6) 包含 1，但不包含 6。

2. range 的常见用法

写法	含义	结果示例
range(5)	从 0 到 4	0,1,2,3,4
range(1, 6)	从 1 到 5	1,2,3,4,5
range(2, 11, 2)	从 2 到 10，每次加 2	2,4,6,8,10

3. 累加求和

任务：计算 1 到 100 的和。

```
total = 0

for i in range(1, 101):
    total = total + i

print("1 到 100 的和是", total)
```

理解：

1. `total = 0` 表示总和从 0 开始。
2. 每次循环，把当前数字 `i` 加到 `total` 中。
3. 循环结束后输出结果。

4. 循环跟踪表

计算 1 到 5 的和时，可以画表理解：

次数 i total 变化

1	1	$0 + 1 = 1$
2	2	$1 + 2 = 3$
3	3	$3 + 3 = 6$
4	4	$6 + 4 = 10$
5	5	$10 + 5 = 15$

五、while 循环

`while` 循环适合“不知道具体重复几次，但知道继续条件”的情况。

1. 输出 1 到 5

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

这个循环包含四个部分：

部分	代码	含义
起点	<code>i = 1</code>	从 1 开始

条件 `i <= 5` 不超过 5 就继续
重复内容 `print(i)` 输出当前数字
变化 `i = i + 1` 每次加 1

2. 避免死循环

如果忘记让 `i` 改变，循环可能一直运行。

错误示例：

```
i = 1

while i <= 5:
    print(i)
```

问题：`i` 一直是 1，条件永远成立。

正确：

```
i = 1

while i <= 5:
    print(i)
    i = i + 1
```

3. 猜数字框架

```
secret = 7
guess = int(input("请猜一个 1 到 10 的数字: "))

while guess != secret:
    print("猜错了，再试一次。")
    guess = int(input("请猜一个 1 到 10 的数字: "))

print("猜对了! ")
```

这里不知道用户会猜几次，所以适合用 `while`。

六、字符串基础

字符串用于保存文字。

```
name = "小明"
message = "hello"
```

1. 字符串长度

```
word = "Python"  
print(len(word))
```

结果：

6

2. 字符串拼接

```
first = "信息"  
second = "科技"  
print(first + second)
```

结果：

信息科技

3. 判断是否包含

```
text = "I like Python"  
if "Python" in text:  
    print("包含 Python")  
else:  
    print("不包含 Python")
```

4. 简单口令检查

```
password = input("请输入口令：")  
if len(password) >= 6:  
    print("口令长度合格")  
else:  
    print("口令太短")
```

提醒：真实密码不能只用长度判断，这里只是练习字符串和条件。

七、列表

列表可以用一个变量保存多个数据。

```
scores = [90, 85, 78, 92]
```

1. 访问列表元素

```
names = ["小明", "小红", "小刚"]
print(names[0])
print(names[1])
print(names[2])
```

注意：Python 列表下标从 0 开始。

2. 遍历列表

```
names = ["小明", "小红", "小刚"]

for name in names:
    print("你好, " + name)
```

3. 添加数据

```
hobbies = []
hobbies.append("篮球")
hobbies.append("绘画")
hobbies.append("编程")
print(hobbies)
```

4. 计算平均分

```
scores = [90, 80, 70]
total = 0

for score in scores:
    total = total + score

average = total / len(scores)
print("平均分是", average)
```

八、函数

函数可以把一组操作封装起来，方便重复使用。

1. 定义和调用函数


```
def say_hello():  
    print("你好! ")  
    print("欢迎学习 Python。")  
  
say_hello()  
say_hello()
```

理解：

- def say_hello(): 定义函数。
- 缩进的代码属于函数内部。
- say_hello() 调用函数。

2. 带参数的函数

```
def greet(name):  
    print("你好, " + name)  
  
greet("小明")  
greet("小红")
```

参数相当于传给函数的数据。

3. 有返回值的函数

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)  
print(result)
```

return 表示把结果返回给调用者。

4. 平均分函数

```
def average(scores):  
    total = 0  
    for score in scores:  
        total = total + score  
    return total / len(scores)  
  
scores = [90, 80, 70]  
print("平均分是", average(scores))
```

九、调试与测试

1. 常见错误

错误	例子	修改方法
漏冒号	<code>if score >= 60</code>	条件后加 :
缩进错误	<code>print()</code> 没有缩进	属于条件或循环内部的代码向右缩进
变量名不一致	<code>score</code> 写成 <code>socre</code>	检查拼写
死循环	<code>while</code> 中变量不变化	添加更新语句
下标越界	列表只有 3 个元素却访问 [3]	记住下标从 0 开始
类型错误	字符串和数字直接相加	用 <code>int()</code> 、 <code>float()</code> 或 <code>str()</code> 转换

2. 测试数据

写程序时至少准备：

类型	例子	用途
正常数据	85 分	检查基本功能
边界数据	60 分、90 分	检查分界线
特殊数据	0 分、100 分	检查极端情况

3. 一次只改一个地方

调试时不要同时改很多行。正确做法：

1. 找到最可能出错的位置。
2. 修改一处。
3. 运行测试。
4. 记录结果。
5. 如果还错，再继续下一处。

十、本节练习

练习 1：成绩等级判断

要求：

1. 输入成绩。
2. 根据分数输出等级。
3. 测试 100、90、89、80、79、60、59、0。

规则：

分数范围	等级
------	----

90 到 100 优秀
80 到 89 良好
60 到 79 合格
0 到 59 需要努力

练习 2：输出偶数

输出 1 到 20 中的偶数。

提示：

```
if i % 2 == 0:  
    print(i)
```

练习 3：猜数字游戏

要求：

1. 设定一个秘密数字。
2. 用户不断输入猜测。
3. 猜错时提示继续。
4. 猜对时输出成功并结束。

拓展：

- 统计猜了几次。
- 提示“太大了”或“太小了”。

练习 4：兴趣列表

要求：

1. 创建一个空列表 hobbies。
2. 让用户输入 3 个兴趣。
3. 把兴趣加入列表。
4. 遍历输出：我喜欢 xxx。

练习 5：平均分函数

要求：

1. 定义函数 average(scores)。
2. 输入或直接创建一个成绩列表。
3. 返回平均分。
4. 输出平均分。

十一、本节达标清单

- ☐ 我能写 if、elif、else 条件判断。
- ☐ 我知道 Python 缩进会影响程序结构。
- ☐ 我能用 for 循环处理固定次数重复。
- ☐ 我能用 while 循环处理条件重复。
- ☐ 我知道如何避免死循环。
- ☐ 我能使用 len()、in 等字符串操作。
- ☐ 我能用列表保存多个数据。
- ☐ 我能遍历列表并计算总和或平均数。
- ☐ 我能定义和调用简单函数。
- ☐ 我能用多组数据测试程序。

完成本节后，你已经具备完成 Python 入门小项目的基础。